assignment1

Jingliang Guo

netid: nxt1510

1. Shuttle: model and methods

State s is the number waiting, from 0 to 200. Action a is 0 (wait) or 1 (dispatch). Capacity is 15. Independent arrivals are uniform over {1, 2, 3, 4, 5}.

```
remaining(s,a) = max(s - 15*a, 0)
cost(s,a) = 100*a + 2*remaining(s,a)
next_state = min(200, remaining(s,a) + arrivals)
gamma = 0.95
```

I charge waiting cost after dispatch, then add arrivals. Arrivals above 200 leave the system without a penalty. An empty shuttle may be sent for cost 100. The finite-horizon terminal cost is zero.

```
Q_t(s,a) = cost(s,a) + gamma * mean(V_(t+1)[next_state])
V_t(s) = min(Q_t(s,0), Q_t(s,1))
```

The mean uses all five arrival outcomes, each with probability 1/5.

Part	Method
(a) Enumeration, T=500	I enumerate both actions and all five arrival outcomes at every state and time. Backward dynamic programming reuses the computed future values: set V_500=0, then solve t=499,...,0. This avoids expanding the full trajectory tree.
(b) Value iteration	Start V=0. Apply the Bellman update using the previous sweep until the largest change is below 1e-6.
(c) Policy iteration	Start with wait everywhere. Evaluate the policy to tolerance 1e-8, then choose the cheaper action at each state. Stop when no action changes.

Part (a) interprets enumeration as checking every state-action-outcome combination, rather than listing complete policies. On ties, (a) and (b) wait; (c) keeps its current action.

Code: main.py, value_iteration.py, policy_iteration/main.py.

2. Shuttle: value functions

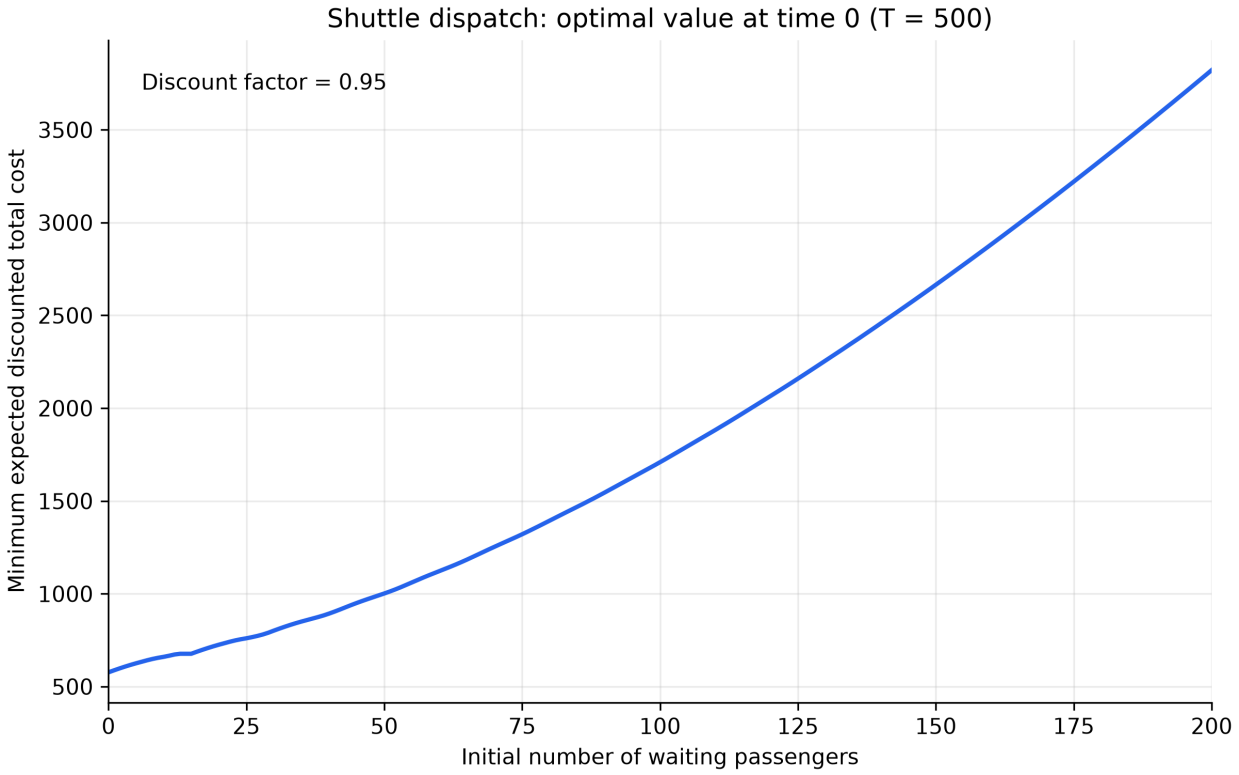


Figure 1. Part (a): minimum discounted cost at time 0, T=500.

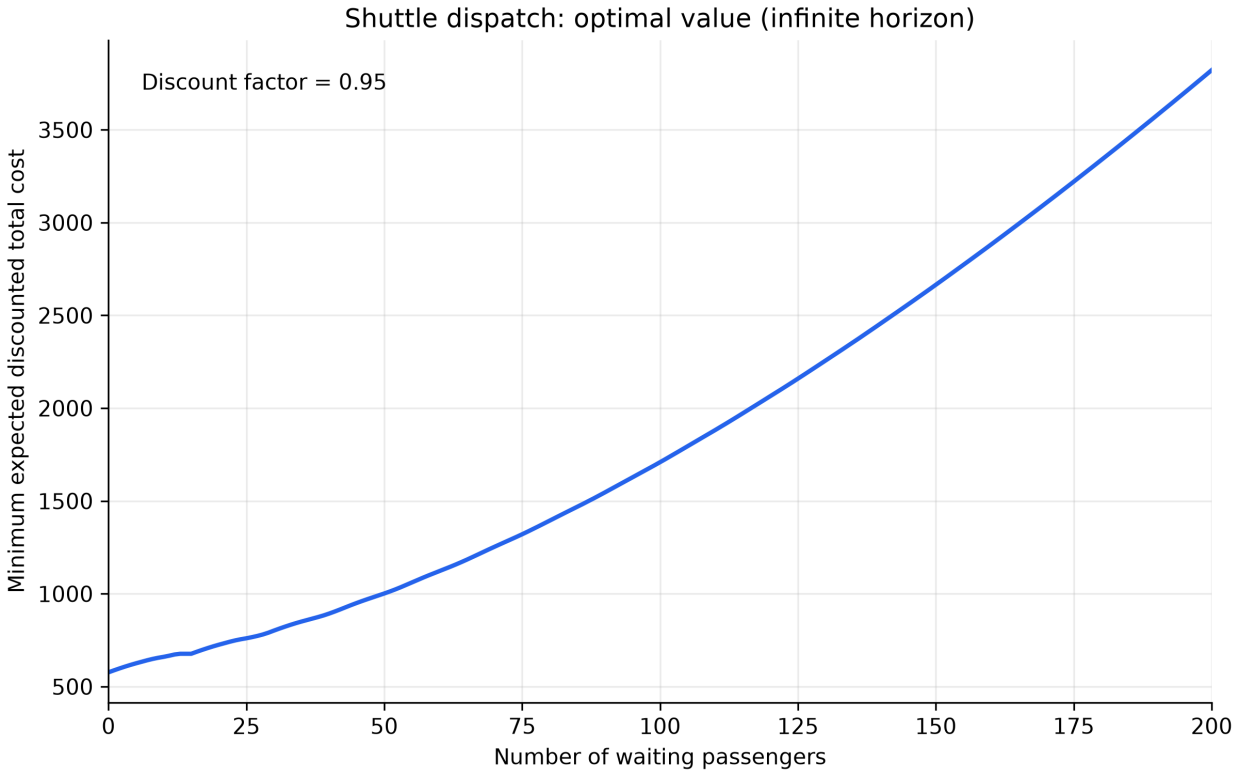


Figure 2. Part (b): minimum discounted cost for the infinite horizon.

More waiting passengers increase cost. The curves nearly overlap because distant costs receive little weight at gamma=0.95. Value iteration took 340 sweeps; its largest difference from T=500 was 1.81e-5.

3. Shuttle: policy and checks

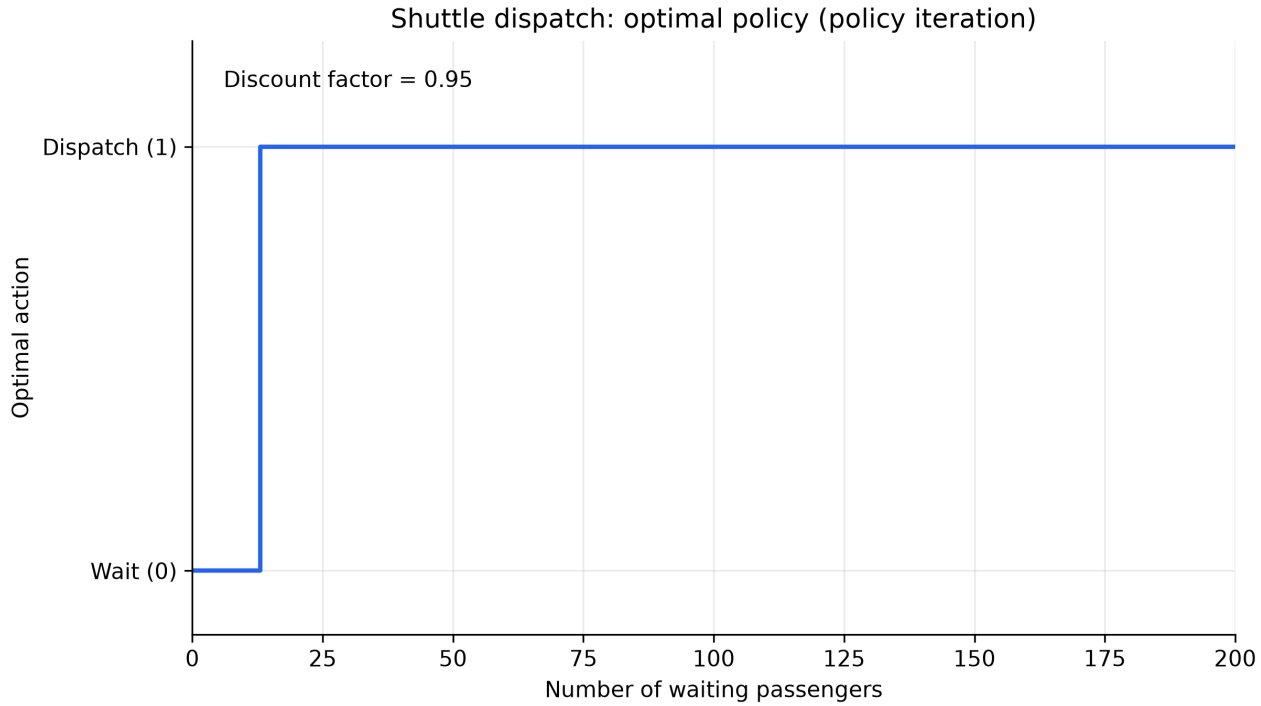


Figure 3. Part (c): wait below 13 passengers; dispatch at 13 or more.

Policy iteration stopped after 6 rounds. All three methods agree on the action at every one of the 201 states.

Test	Expected result	Observed result
gamma=0	Ignore future costs. Dispatch costs at least 70 extra, so always wait; V(s)=2s.	201/201 actions correct; value error 0.
Empty / partial / full	Dispatch at s=0 or 15 costs 100; at s=16 it costs 102.	All 7 hand-calculated cost checks pass.
Queue cap	From 198 remaining, next states are 199,200,200,200,200; P(200)=4/5.	All 3 expectation checks pass.
Always wait	At the cap, V(200)=400/(1-0.95)=8000. Values at 198 and 199 also have analytic solutions.	Largest error 1.89e-7; tolerance 2e-7.
Compare methods	T=500 should be close to the infinite-horizon solutions.	201/201 actions agree; largest value gap 1.81e-5.

The analytic checks provide independent answers; agreement between implementations alone could miss a shared error.

Run: python policy_iteration/validate.py --experiment all. Results: policy_iteration/validation_*.json.

4. Policy gradient: method

I use REINFORCE with one ReLU hidden layer. The network outputs two action logits. Actions are sampled during training and testing. Adam updates the policy once per episode.

Setting	CartPole-v1	ALE/Pong-v5
Input / actions	4 state values; actions 0/1	6400 frame-difference pixels; actions 2/3
Network	4 -> 32 -> 2	6400 -> 128 -> 2
Discount / Adam rate	0.95 / 0.01	0.99 / 0.001
Training episodes	2000 per setting	5000 per setting
Initialization	PyTorch default	He hidden weights; zero biases; orthogonal output, gain 0.01
Device	CPU	One RTX A6000 on Deepdish

For Pong, I use the supplied crop, downsampling and binary preprocessing, then subtract consecutive frames to show motion. The previous frame is zero at reset. I retain v5 defaults: frameskip 4 and sticky-action probability 0.25.

```
G[T] = 0
G[t] = r[t] + gamma * G[t+1]
loss = -sum_t gamma^t * (G[t] - b) * log pi(a[t] | s[t])
```

Returns continue across Pong points. Gradients are summed in 256-state chunks before one Adam step. Episodes end on termination or truncation, with no bootstrap value.

Part 2: historical EMA baseline

```
b = 0
# Each episode: use the old b in the loss above.
update_policy()
reference = sum_t(gamma^t * G[t]) / sum_t(gamma^t)
b = 0.9*b + 0.1*reference
```

Part 1 uses b=0 throughout. Part 2 uses a scalar baseline from previous episodes, independent of the current action. Both settings use the same starting weights and training budget for each game.

Evaluation

Each trained policy is frozen and tested for 500 episodes. Scores are undiscounted reward sums; standard deviations use ddof=0. Training plots show episode rewards and a trailing 100-episode average, starting at episode 100.

Game	Baseline	Test mean	Test std. dev.
CartPole	none	123.094	36.230
CartPole	ema	369.722	65.631
Pong	none	-18.374	2.743
Pong	ema	-18.786	2.709

5. CartPole: no baseline

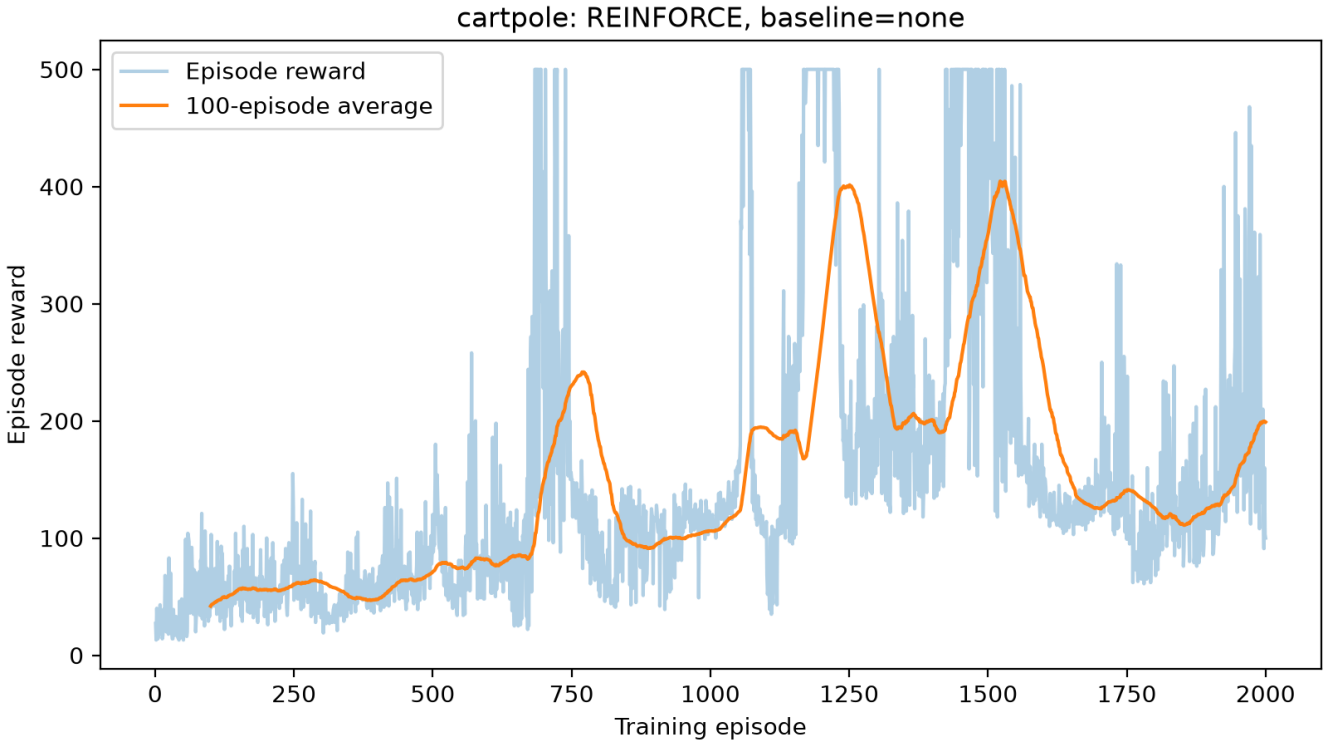


Figure 4. Training rewards and the 100-episode moving average.

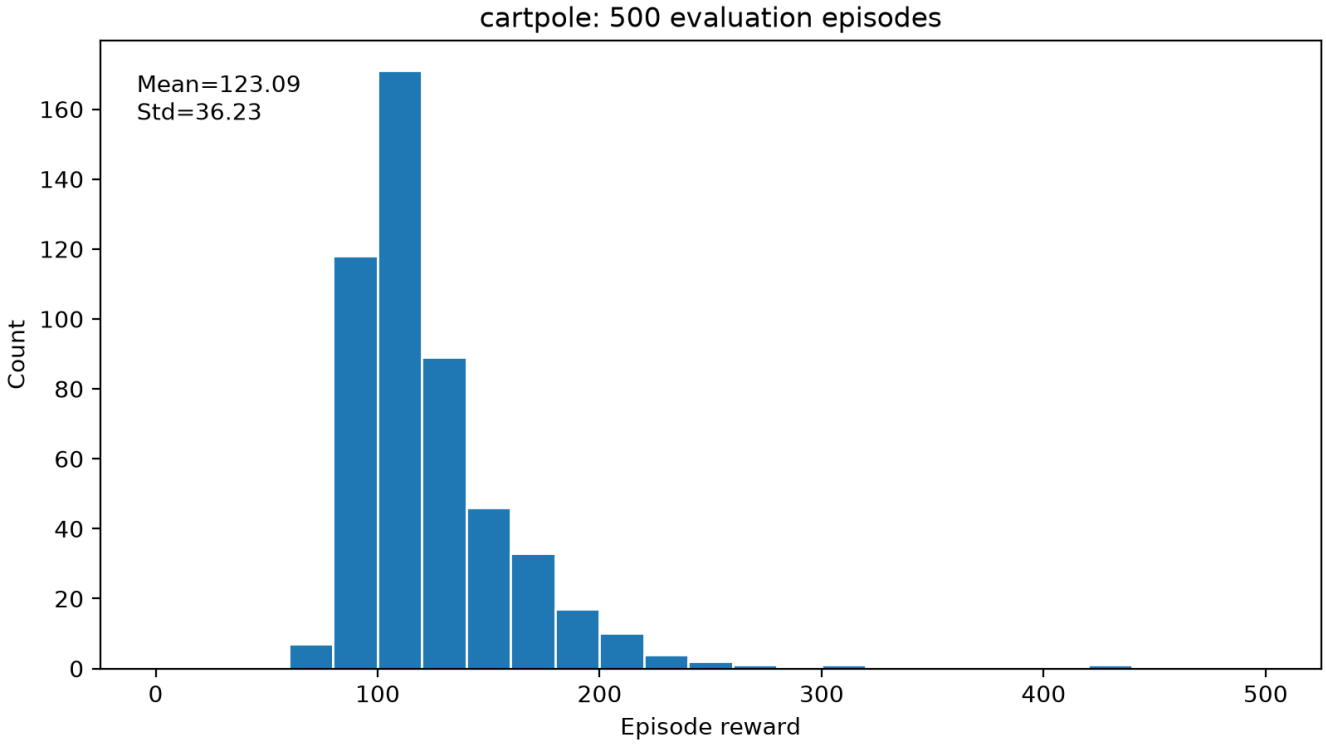


Figure 5. Reward distribution over 500 frozen-policy test episodes.

The mean test score is 123.094. Learning is unstable, and the policy rarely reaches the 500-step limit.

6. CartPole: EMA baseline

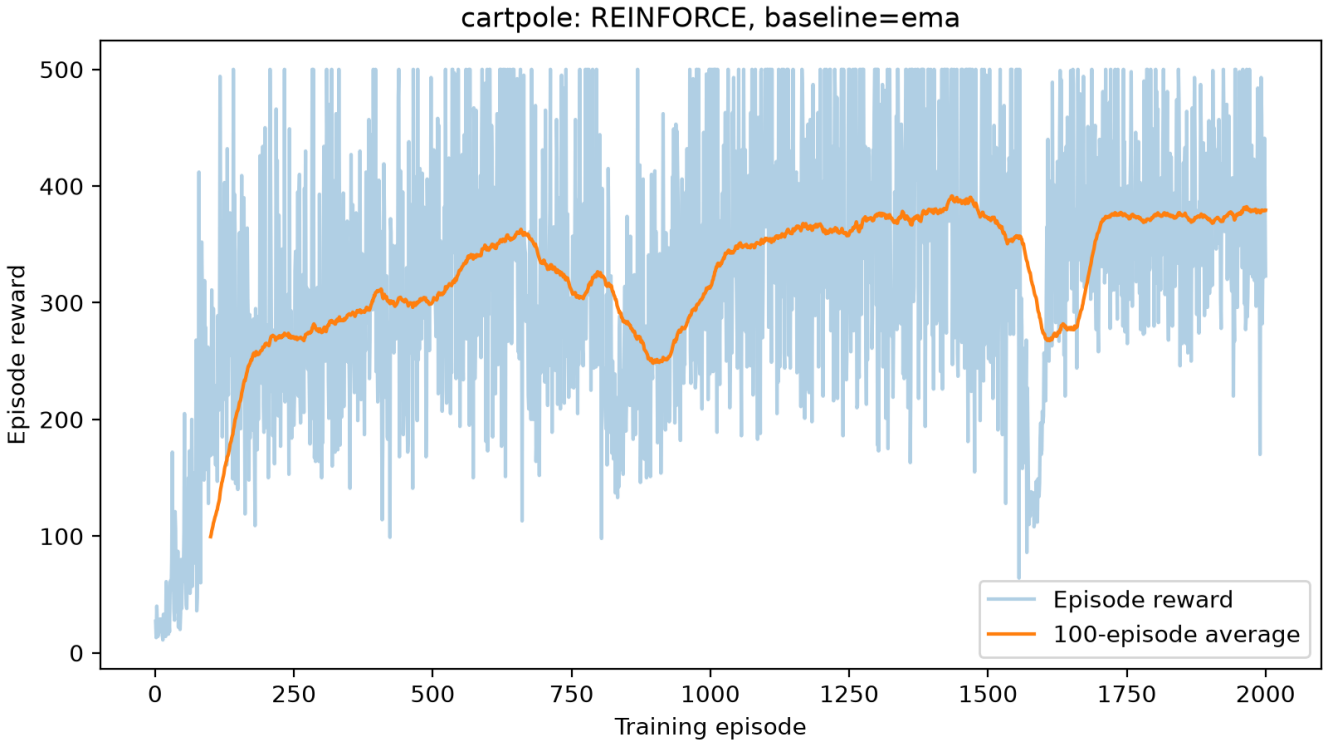


Figure 6. Training rewards and the 100-episode moving average.

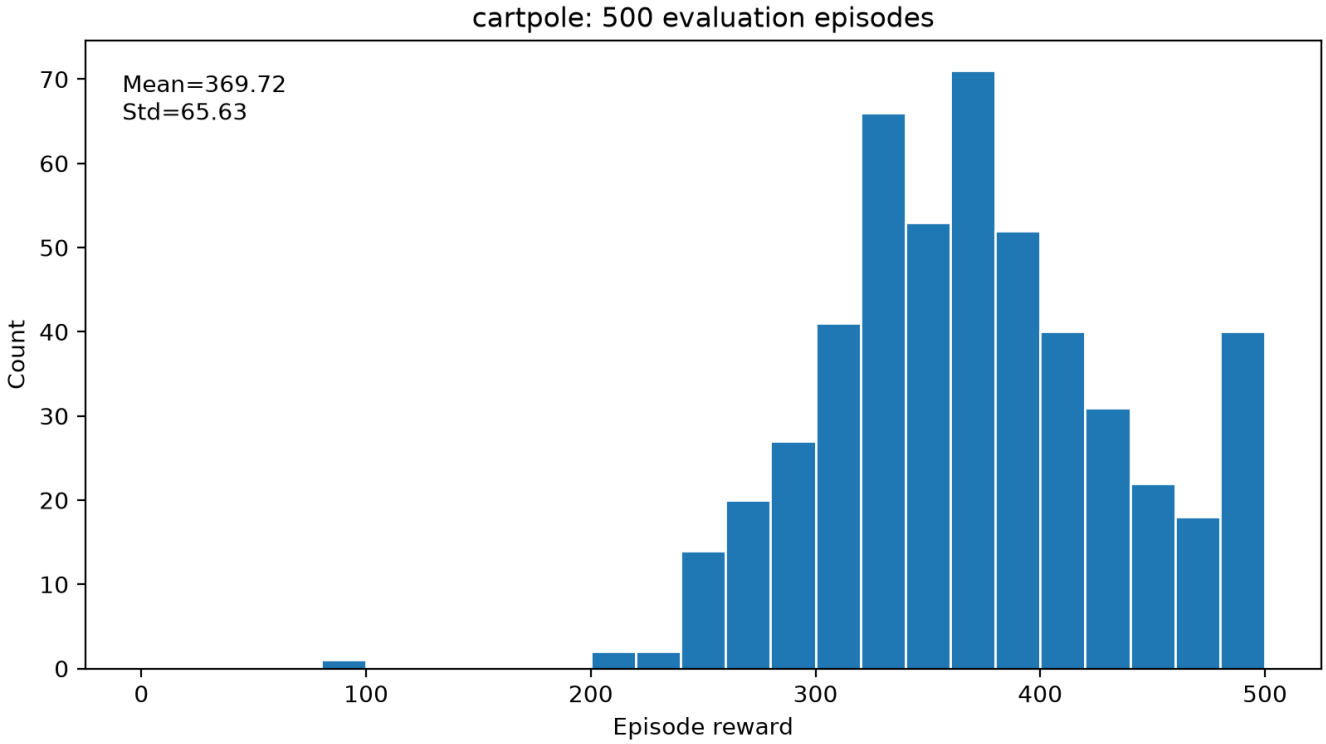


Figure 7. Reward distribution over 500 frozen-policy test episodes.

The mean test score is 369.722, an increase of 246.628 over no baseline in this pair of runs.

7. Pong: no baseline

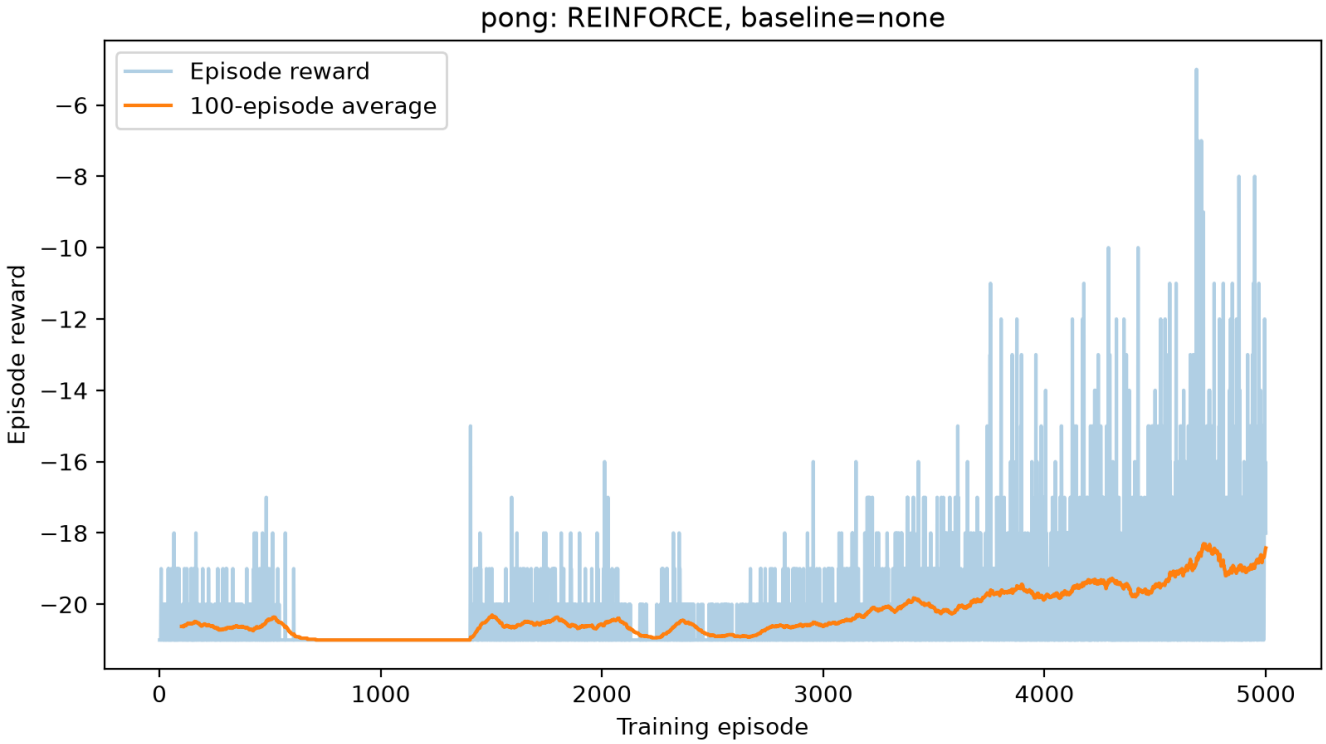


Figure 8. Training rewards and the 100-episode moving average.

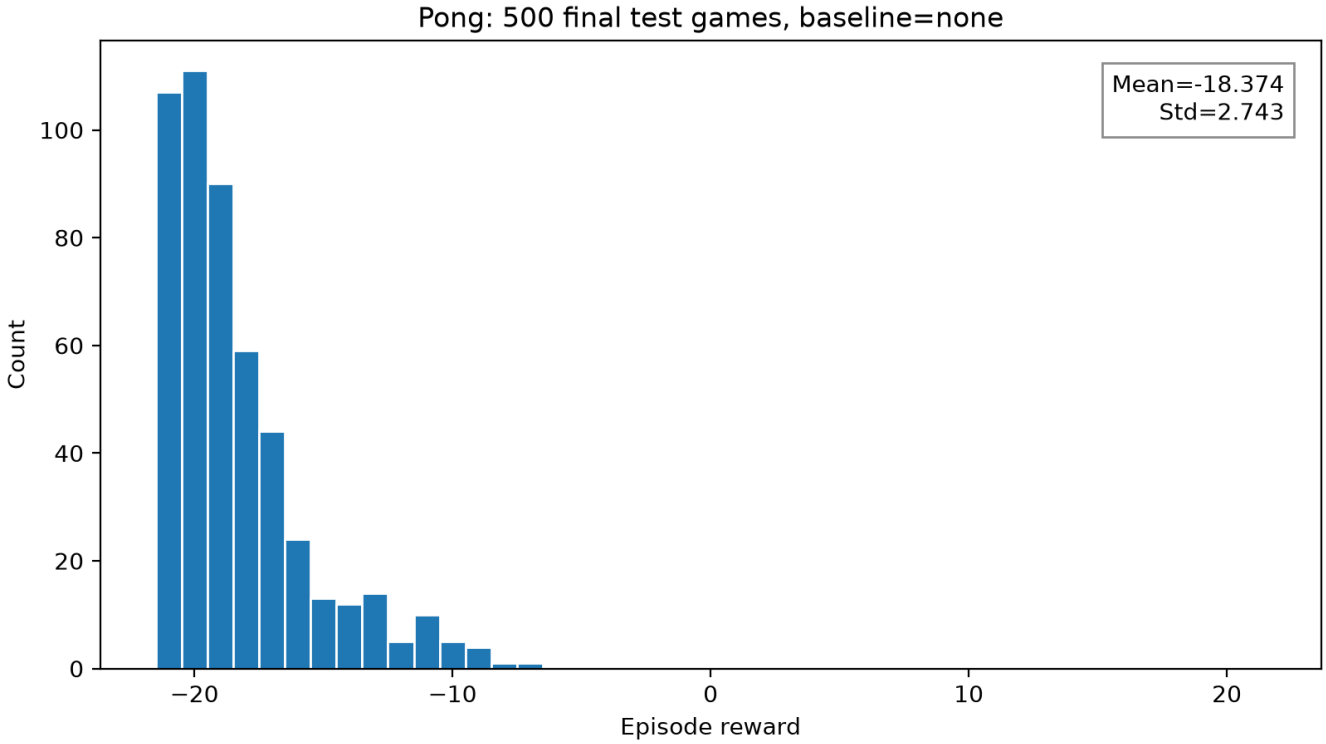


Figure 9. Reward distribution over 500 frozen-policy test episodes.

The mean test score is -18.374. Scores improve late in training, but all 500 test games are losses.

8. Pong: EMA baseline

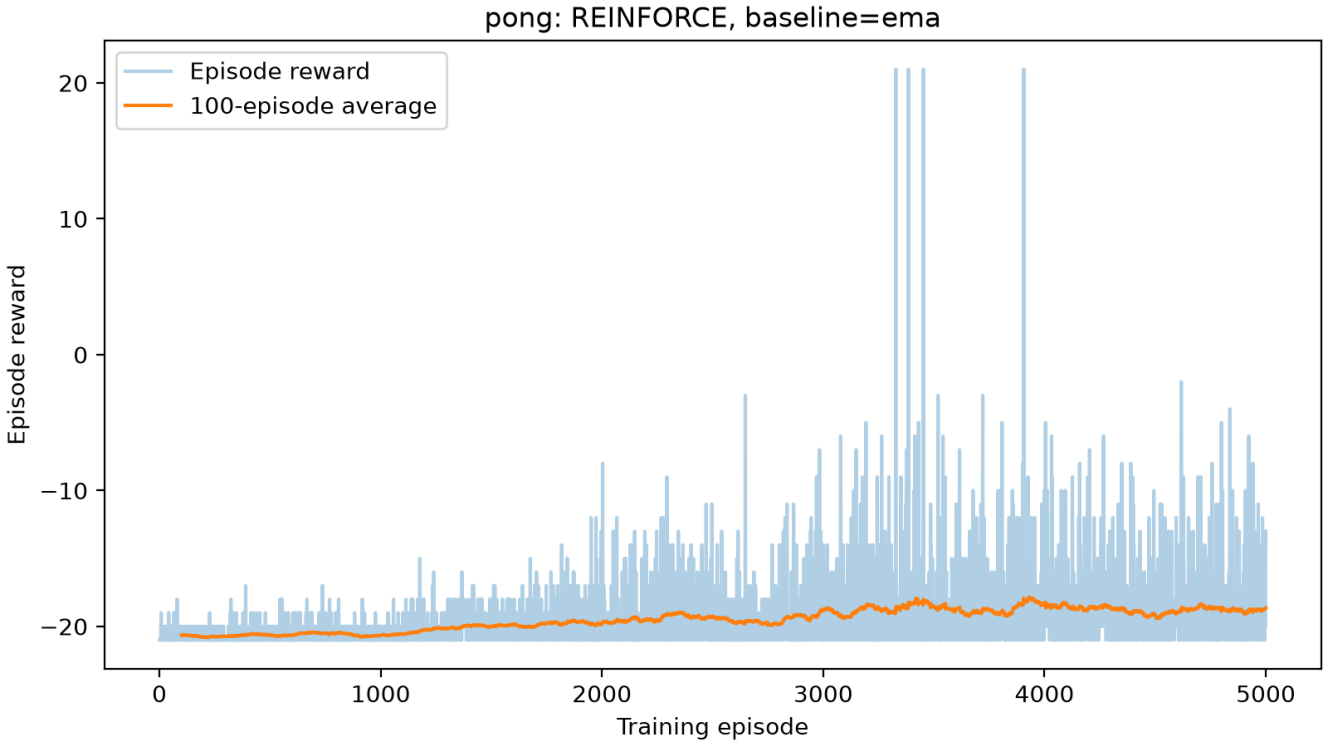


Figure 10. Training rewards and the 100-episode moving average.

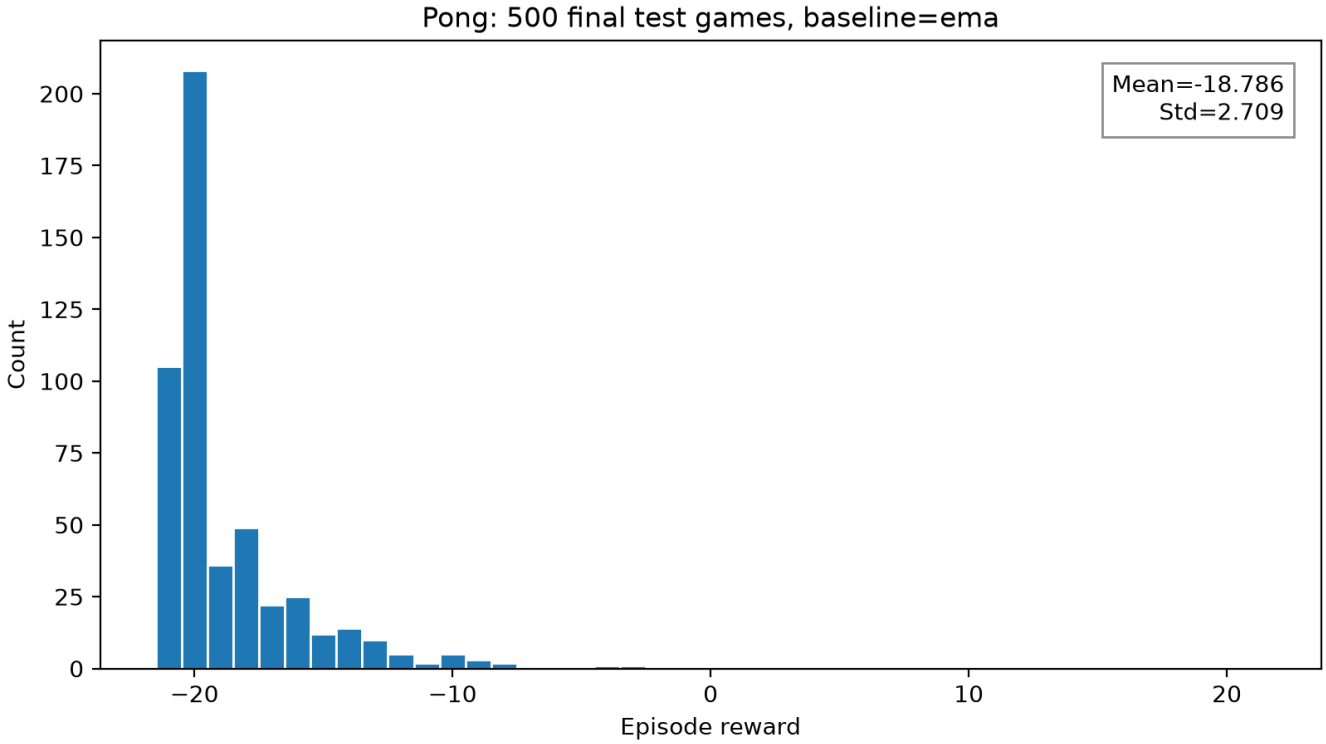


Figure 11. Reward distribution over 500 frozen-policy test episodes.

The mean test score is -18.786. EMA improves earlier in training, but all 500 test games are losses.

9. Policy gradient: validation

Check	Method and expected result	Result
Exact gradient	Enumerate all 4 two-step trajectories. Compare expected REINFORCE gradients with direct differentiation of J=p(0)+0.95*2*p(1), for b=0 and b=0.8.	Maximum errors 8.94e-8 and 1.49e-8, below 1e-6. Checks the gradient sign, discounts and baseline invariance.
Chunked gradient	Compare chunked and full gradients on the same 887-step Pong trajectory.	Relative error 8.46e-8. A small gradient step also lowers the fixed-trajectory loss.
Pong replay	Freeze the policy and repeat the environment and action seeds.	Both 946-step action/reward traces match; score -21.

Does the baseline reduce gradient variance?

For each game, freeze an untrained policy and estimate b from 20 separate pilot episodes. On each new trajectory, compute gradients both with and without b. Measure the sum of sample variances across all gradient components.

Game	Paired episodes	No baseline	EMA	EMA / none
CartPole	40	891.86	151.81	0.1702
Pong	20	609.03	107.61	0.1767

Variance falls by about 83% for CartPole and 82% for Pong in these checks. The Pong check uses a seed-0, default-initialized 128-unit policy, not either final trained model. This supports variance reduction at those fixed policies; it does not guarantee higher scores.

Edge cases

Pong preprocessing preserves the original image and produces binary frames. Network actions 0/1 map to environment actions 2/3. Frame history resets each episode. Both termination flags are handled; log probabilities use logits, and non-finite losses are rejected.

In the saved Pong replay, the first 10 actions are 3,3,3,3,2,2,3,3,2,2. All 10 rewards are zero because no point has been scored.

Evidence: policy_gradient/evidence.json. Validation commands are on page 11.

10. Experiments and discussion

Pong experiment	Observation	Conclusion
Learning rate	After 100 episodes: lr=0.01 scored -20.90; lr=0.0001 scored -20.55 on 20 validation games. The former had 72.2% saturated probe actions.	The suggested rate was unstable. I used 0.001 for the final runs.
Hidden width	At 300 episodes, 50-game means for 32/64/128/256 units were -20.42/-20.40/-20.32/-20.84.	More units did not consistently help.
Initialization	128 units with He/small-output initialization scored -20.70 at 300 episodes.	No early advantage; retained for the longer run.
Training length	That no-baseline model improved from -20.70 at 300 episodes to -18.60 at 5000 on 50 validation games.	Longer training helped.
EMA baseline	At 5000 episodes, validation means were -18.60 without and -18.42 with EMA; the difference interval crossed zero.	No clear validation advantage.
Historical loss variant	Omitting the outer gamma^t scored -19.80 after 600 episodes on 50 validation games.	This changes the estimator. Final runs retain gamma^t.

Final comparison and limits

CartPole improves with EMA in this run. For Pong, EMA minus no baseline is -0.412 points; a paired bootstrap over 500 test seeds gives a 95% interval of [-0.706, -0.130]. Mean discounted return favors EMA (-0.204 vs -0.264), but whole-game score does not. These results use one training seed, so they do not establish which method is better across retraining.

Pong rewards are sparse. At gamma=0.99, rewards 500 steps away receive weight 0.00657. Exploration also drops: 40.5% of no-baseline and 73.4% of EMA probe actions have probability above 99%. These may explain weak scores, but are not causal tests. The reused tuning set also limits model selection.

With three more months

Period	Experiments and measurements
Month 1	Repeat all settings with at least 5 training seeds and report uncertainty. Test shuttle deterministic/zero arrivals, free dispatch, changed capacity and cap rules against exact solutions.
Month 2	Compare learning rates, batch sizes, widths and baseline types at matched budgets. Measure scores, learning speed and paired gradient variance at several checkpoints.
Month 3	Test unusual and near-terminal states, deterministic diagnostics and longer games. Freeze selected settings and evaluate on fresh test seeds; report failures and confidence intervals.

11. Code and reproduction

GitHub username: jingliangguo2027
Repository: mlds-assignment-1
https://github.com/jingliangguo2027/mlds-assignment-1

Start in hw1. Install PyTorch for your CPU or CUDA setup, then run:

```
python -m pip install -r requirements.txt
python -m pip install reportlab
python main.py
python value_iteration.py
python policy_iteration/main.py
python policy_iteration/validate.py --experiment all
```

Train and test

Run each command with BASELINE replaced by none, then ema. Use new output folders.

```
cd policy_gradient
python train.py --game cartpole --baseline BASELINE \
  --episodes 2000 --eval-episodes 500 --lr 0.01 \
  --device cpu --output results/new_cartpole_BASELINE

CUDA_VISIBLE_DEVICES=0 python train.py --game pong \
  --baseline BASELINE --episodes 5000 --eval-episodes 500 \
  --hidden 128 --initialization he_small_output --lr 0.001 \
  --device cuda --output results/new_pong_BASELINE
```

The runner keeps the original method and seeds: Pong action seeds are 10000+episode and environment seeds are episode. Exact results can vary across devices or library versions. --resume continues a new-format checkpoint; old checkpoints remain in the external backup.

Validation commands (from policy_gradient)

```
python validate.py
CUDA_VISIBLE_DEVICES=0 python validate.py --extra pong-variance --device cuda
CUDA_VISIBLE_DEVICES=0 python validate.py --extra chunk --device cuda
```

The default checks exact gradients, Pong preprocessing/replay and CartPole gradient variance. The extra modes check Pong variance and chunked gradients against a full-trajectory gradient. They print fresh results without replacing the saved evidence.

Saved data and environment

Raw training and test scores, settings and plots are in policy_gradient/results/. Evidence and selected tuning summaries are consolidated in policy_gradient/evidence.json. Model files are not in Git. Final test seeds: 1000000-1000499; tuning seeds: 200000-200049.

Recorded environment: PyTorch 2.14.1 (Pong: +cu126), Gymnasium 1.4.0, NumPy 2.5.4 and ALE 0.12.1. Pong used one GPU. From hw1, run python build_report.py to validate the four recorded score sets and rebuild this PDF.

Historical tuning scripts and original checkpoint formats remain available in Git history (e8ec206). The current code keeps the assignment methods and merges the necessary validation checks.